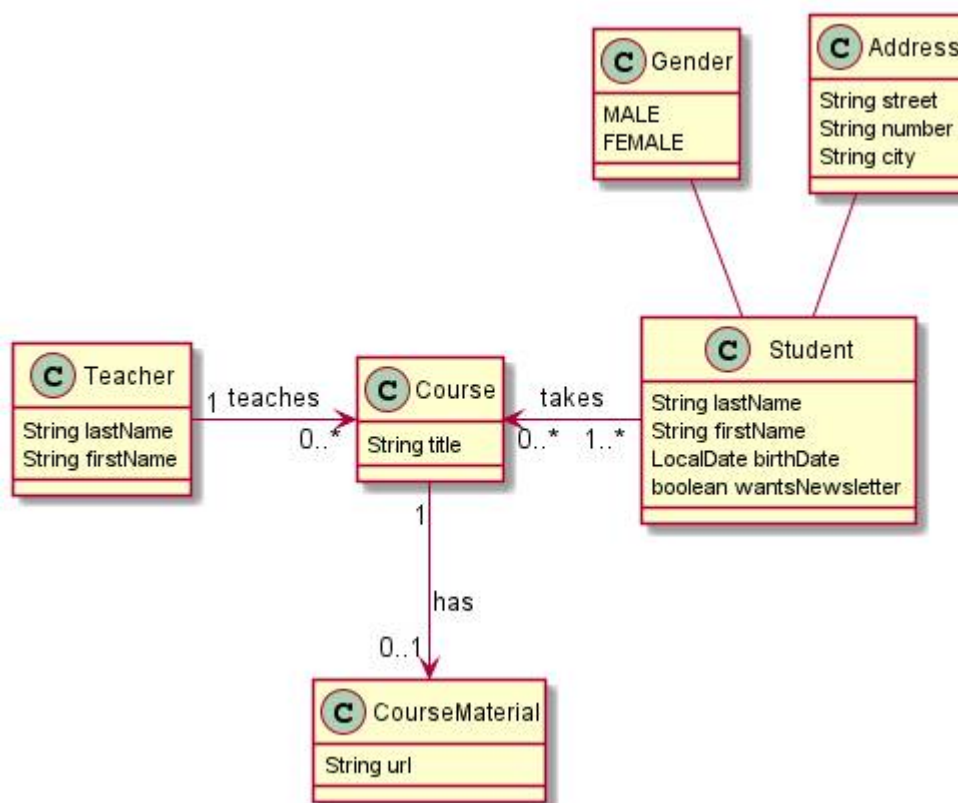


Chapter 5: Relationship

- Introduction
- One to Many
- Many to One
- One to One
- Many to Many

Relationships

First of all, let's define a relationship. If we look at our class diagram we can see a few relationships:



Teachers and courses - students and courses - courses and course materials.

There are also connections between students and addresses, but they aren't considered relationships. This is because an Address is not an entity (i.e. it is not mapped to a table of its own). So, as far as JPA's concerned, it's not a relationship.

There are a few types of relationships:

1. One-to-Many
2. Many-to-One
3. One-to-One
4. Many-to-Many

One-to-Many/Many-to-One

We'll get started with the One-to-Many and Many-to-One relationships, which are closely related. They are the opposite sides of the same coin.

What's a One-to-Many relationship?

As its name implies, it's a relationship that links one entity to many other entities.

In our example, this would be a Teacher and their Courses. A teacher can give multiple courses, but a course is given by only one teacher (that's the Many-to-One perspective - many courses to one teacher).

Another example could be on social media - a photo can have many comments, but each of those comments belongs to that one photo.

Before diving into the details of how to map this relationship, let's create our entities:

```
@Entity
public class Teacher {
    private String firstName;
    private String lastName;
}
```

```
@Entity
public class Course {
    private String title;
}
```

Now, the fields of the Teacher class should include a list of courses. We will annotate it with a @OneToMany annotation:

```
@OneToMany
private List<Course> courses;
```

How does JPA reflect this relationship in the database? Generally, for this type of relationship, we must use a foreign key in a table.

JPA does this for us, given our input on how it should handle the relationship. This is done via the @JoinColumn annotation:

```
@OneToMany
@JoinColumn(name = "TEACHER_ID", referencedColumnName = "ID")
private List<Course> courses;
```

Using this annotation will tell JPA that the COURSE table must have a foreign key column TEACHER_ID that references the TEACHER table's ID column.

Owning Side and Bidirectionality

In the previous example, the Teacher class is called the owning side of the One-To-Many relationship. This is because it defines the join column between the two tables.

We could've made Course the owning side of the relationship by mapping the Teacher field with @ManyToOne in the Course class instead:

```
@ManyToOne
@JoinColumn(name = "TEACHER_ID", referencedColumnName = "ID")
private Teacher teacher;
```

There's no need to have a list of courses in the Teacher class now. The relationship would've worked the opposite way:

The Course is called the referencing side in that relationship.

This time, we used the @ManyToOne annotation, in the same way we used @OneToMany.

Note: It's a good practice to put the owning side of a relationship in the class/table where the foreign key will be held.

So, in our case this second version of the code is better. But, what if we still want our Teacher class to offer access to its Course list?

We can do that by defining a bidirectional relationship:

```
@Entity
public class Teacher {
    // ...

    @OneToMany(mappedBy = "teacher")
    private List<Course> courses;
}

@Entity
public class Course {
    // ...

    @ManyToOne
    @JoinColumn(name = "TEACHER_ID", referencedColumnName = "ID")
    private Teacher teacher;
}
```

We keep our @ManyToOne mapping on the Course entity. However, we also map a list of Courses to the Teacher entity.

What's important to note here is the use of the mappedBy flag in the @OneToMany annotation on the referencing side.

Without it, we wouldn't have a two-way relationship. We'd have two one-way relationships. Both entities would be mapping foreign keys for the other entity.

With it, we're telling JPA that the field is already mapped by another entity. It's mapped by the teacher field of the Course entity.

Eager VS Lazy Loading

Another thing worth noting is eager and lazy loading. With all our relationships mapped, it's wise to avoid impacting the software's memory by putting too many entities in it if unnecessary.

Imagine that Course is a heavy object, and we load all Teacher objects from the database for some operation. We don't need to retrieve or use the courses for this operation, but they're still being loaded alongside the Teacher objects.

This can be devastating for the application's performance. Technically, this can be solved by using the Data Transfer Object Design Pattern and retrieving Teacher information without the courses.

However, this can be a massive overkill if all we're gaining from the pattern is excluding the courses.

Thankfully, JPA thought ahead and made One-to-Many relationships load lazily by default.

This means that the relationship won't be loaded right away, but only when and if actually needed.

In our example, that would mean until we call on the Teacher#courses method, the courses are not being fetched from the database.

By contrast, Many-to-One relationships are eager by default, meaning the relationship is loaded at the same time the entity is.

We can change these characteristics by setting the fetch argument of both annotations:

```
@OneToMany(mappedBy = "teacher", fetch = FetchType.EAGER)
private List<Course> courses;
```

```
@ManyToOne(fetch = FetchType.LAZY)
private Teacher teacher;
```

That would inverse the way it worked initially. Courses would be loaded eagerly, as soon as we load a Teacher object. By contrast, the teacher wouldn't be loaded when we fetch courses if it's unneeded at the time.

Optionality

A relationship may be optional or mandatory.

Considering the One-to-Many side - it is always optional, and we can't do anything about it. The Many-to-One side, on the other hand, offers us the option of making it mandatory.

By default, the relationship is optional, meaning we can save a Course without assigning it a teacher:

```
Course course = new Course("C# 101");
session.save(course);
```

Now, let's make this relationship mandatory. To do that, we'll use the optional argument of the @ManyToOne annotation and set it to false (it's true by default):

```
@ManyToOne(optional = false)
@JoinColumn(name = "TEACHER_ID", referencedColumnName = "ID")
private Teacher teacher;
```

Thus, we can no longer save a course without assigning a teacher to it:

```
Course course = new Course("C# 101");
assertThrows(Exception.class, () -> session.save(course));
```

But if we give it a teacher, it works fine again:

```
Teacher teacher = new Teacher();
teacher.setLastName("Doe");
teacher.setFirstName("Will");

Course course = new Course("C# 101");
course.setTeacher(teacher);
```

```
session.save(course);
```

Well, at least, it would seem so. If we had run the code, an exception would've been thrown:

```
javax.persistence.PersistenceException: org.hibernate.PersistentObjectException: detached entity passed to persist: com.fupro.clients.stackabuse.jpa.domain.Course
```

Why is this? We've set a valid Teacher object in the Course object we're trying to persist. However, we haven't persisted the Teacher object before trying to persist the Course object.

Thus, the Teacher object isn't a managed entity. Let's fix that and try again:

```
Teacher teacher = new Teacher();
teacher.setLastName("Doe");
teacher.setFirstName("Will");
session.save(teacher);

Course course = new Course("C# 101");
course.setTeacher(teacher);

session.save(course);
```

Running this code will persist both entities and preserve the relationship between them.

Cascading Operations

However, we could've done another thing - we could've cascaded, and thus propagated the persistence of the Teacher object when we persist the Course object.

This makes more sense and works the way we'd expect it to like in the first example which threw an exception.

To do this, we'll modify the cascade flag of the annotation:

```
@ManyToOne(optional = false, cascade = CascadeType.PERSIST)
@JoinColumn(name = "TEACHER_ID", referencedColumnName = "ID")
private Teacher teacher;
```

This way, Hibernate knows to persist the needed object in this relationship as well.

There are multiple types of cascading operations: PERSIST, MERGE, REMOVE, REFRESH, DETACH, and ALL (that combines all the previous ones).

We can also put the cascade argument on the One-to-Many side of the relationship, so that operations are cascaded from teachers to their courses as well.

Example: test8

Pre-Requirement: Create a database by the **hib** in **MySQL 8**

1. test8\pom.xml
2. src\main\java\hibernate.cfg.xml
3. com.vedisoft.hibernate.test8.utils
 - **HibernateUtil.java**
4. com.vedisoft.hibernate.test8.pojos
 - **Course.java** @Entity
 - **Teacher.java** @Entity
5. com.vedisoft.hibernate.test8.daos
 - **ICourseDao.java**
 - **CourseDao.java**
 - **ITeacherDao.java**
 - **TeacherDao.java**
6. com.vedisoft.hibernate.test8
 - **App.java**

One-to-One

Now that we've set up the foundations of relationship mapping in JPA through One-to-Many / Many-to-One relationships and their settings, we can move on to One-to-One relationships.

This time, instead of having a relationship between one entity on one side and a bunch of entities on the other, we'll have a maximum of one entity on each side.

This is, for example, the relationship between a Course and its CourseMaterial. Let's first map CourseMaterial, which we haven't done yet:

```
@Entity
public class CourseMaterial {
    @Id
    private Long id;
    private String url;
}
```

The annotation for mapping a single entity to a single other entity is @OneToOne.

Before setting it up in our model, let's remember that a relationship has an owning side - preferably the side which will hold the foreign key in the database.

In our example, that would be CourseMaterial as it makes sense that it references a Course (though we could go the other way around):

```
@OneToOne(optional = false)
@JoinColumn(name = "COURSE_ID", referencedColumnName = "ID")
private Course course;
```

There is no point in having material without a course to encompass it. That's why the relationship is not optional in that direction.

Speaking of direction, let's make the relationship bidirectional, so we can access the material of a course if it has one. In the Course class, let's add:

```
@OneToOne(mappedBy = "course")
private CourseMaterial material;
```

Here, we're telling Hibernate that the material within a Course is already mapped by the course field of the CourseMaterial entity.

In addition to making the relationship bidirectional, we could also add cascading operations or make entities load eagerly or lazily.

Example: test9

Pre-Requisite: Create a database by the **hib** in **MySQL 8**

1. test9\pom.xml
2. src\main\java\hibernate.cfg.xml
3. com.vedisoft.hibernate.test9.utils
 - HibernateUtil.java
4. com.vedisoft.hibernate.test9.pojos
 - Course.java @Entity
 - CourseMaterial.java @Entity
5. com.vedisoft.hibernate.test9.daos
 - ICourseDao.java
 - CourseDao.java
 - ICourseMaterialDao.java
 - CourseMaterialDao.java
6. com.vedisoft.hibernate.test9
 - App.java

Many-to-Many

Effectively, in a database, a Many-to-Many relationship involves a middle table referencing both other tables.

Luckily for us, JPA does most of the work, we just have to throw a few annotations out there, and it handles the rest for us.

So, for our example, the Many-to-Many relationship will be the one between Student and Course instances as a student can attend multiple courses, and a course can be followed by multiple students.

In order to map a Many-to-Many relationship we'll use the `@ManyToMany` annotation. However, this time around, we'll also be using a `@JoinTable` annotation to set up the table that represents the relationship:

```
@ManyToMany
@JoinTable(
    name = "STUDENTS_COURSES",
    joinColumns = @JoinColumn(name = "COURSE_ID", referencedColumnName = "ID"),
    inverseJoinColumns = @JoinColumn(name = "STUDENT_ID", referencedColumnName = "ID")
)
private List<Student> students;
```

The annotation takes a few parameters. First of all, we must give the table a name. We've chosen it to be `STUDENTS_COURSES`.

After that, we'll need to tell Hibernate which columns to join in order to populate `STUDENTS_COURSES`. The first parameter, `joinColumns` defines how to configure the join column (foreign key) of the owning side of the relationship in the table. In this case, the owning side is a Course.

On the other hand, the `inverseJoinColumns` parameter does the same, but for the referencing side (Student).

We can still map the relationship as bidirectional the same way we did for the previous relationships.

```
@ManyToMany(mappedBy = "students")
private List<Course> courses;
```

We can also cascade operations as well as define if entities should load lazily or eagerly (Many-to-Many relationships are lazy by default).

Example: test10

Pre-Requirement: Create a database by the **hib** in **MySQL 8**

1. test10\pom.xml
2. src\main\java\hibernate.cfg.xml
3. com.vedisoft.hibernate.test10.utils
- **HibernateUtil.java**
4. com.vedisoft.hibernate.test10.pojos
- **Course.java** @Entity

- **Student.java** @Entity
- 5. com.vedisoft.hibernate.test10.daos
 - **ICourseDao.java**
 - **CourseDao.java**
 - **IStudentDao.java**
 - **StudentDao.java**
- 6. com.vedisoft.hibernate.test10
 - **App.java**